

ORBITAL 2025: NUS School of Computing



NUS
National University
of Singapore

Team Name:

NUS ScheduleBoss

Done by:

Tran Viet Khoa
Duong Hoang Minh

Proposed Level of Achievement:

Apollo 11

Links:

Link to Website: <https://michael-coding06.github.io/NUS-ScheduleBoss/>

- Once you've clicked the Supabase authentication link in your email, please return to the site and log in. Thank you 🙏
- Click log out to save your data for the next time you log in

Link to Github Repo (*for our advisor only*):

Project Motivation

As NUS students, we constantly face the challenge of creating organized, balanced schedules that accommodate both academic and personal commitments. Traditional scheduling approaches often leave us juggling multiple tools and platforms, manually coordinating class times with personal tasks, CCAs, work commitments, and social activities.

While existing platforms like NUSMods excel at basic timetable creation, they lack the intelligence to provide personalized scheduling assistance or integrate non-academic commitments. Students are left to manually assess module workloads, coordinate conflicting time slots, and maintain separate systems for academic and personal planning. This fragmented approach often leads to scheduling conflicts, poor time management, and difficulty maintaining work-life balance.

NUS ScheduleBoss bridges this gap by introducing intelligent, conversational scheduling assistance specifically designed for the NUS ecosystem. Our platform transforms schedule management from a tedious manual process into an intuitive, AI-powered experience that seamlessly integrates academic modules with personal commitments in one unified interface.

Aim

Through this project, we aim to achieve several meaningful outcomes that will not only contribute to our personal and professional growth but also provide tangible value to the NUS student community.

- First and foremost, we are excited to gain hands-on experience in full-stack software development and machine learning. By working on both the frontend and backend components of the platform, as well as integrating intelligent recommendation systems, we are immersing ourselves in the complete product development lifecycle. This includes user interface design, database management, server deployment, and algorithm optimization, skills that are highly sought after in today's tech-driven job market. We believe that this practical experience will serve as a strong foundation for our future careers in technology, product management, or research.
- In parallel, we are committed to building a functional and user-centric web-based tool that directly addresses a common pain point among NUS students: the challenge of organizing their academic schedules in a smart and balanced way. Our goal is to design a platform that goes beyond static timetable planning, one that dynamically adjusts to user preferences, course requirements, and external commitments, offering a personalized scheduling experience that saves time and reduces stress.

User Stories

1. **As a NUS student who finds timetable planning tedious**, I want an intelligent assistant that helps me arrange my modules through natural conversation, so I can quickly create optimized schedules without manually checking for time conflicts.
2. **As a student managing multiple commitments**, I want to add both academic modules and personal tasks (like CCA meetings, work, and study sessions) to one unified timetable, so I can visualize my entire week and avoid scheduling conflicts.
3. **As a student choosing between different modules**, I want access to authentic reviews and experiences from fellow NUS students, so I can

make informed decisions about course selection based on real student feedback rather than just official descriptions.

4. **As a student concerned about workload balance**, I want insights into module difficulty and time commitments from peer reviews, so I can better understand what to expect and plan my semester accordingly.
5. **As a student who values organization**, I want a clean, visual calendar interface where I can easily toggle between different types of commitments (modules, tasks, meetings, sports), so I can focus on specific aspects of my schedule when needed.

Tech Stack

To build ScheduleBoss, our personalized scheduling assistant for NUS students, we've selected a modern, scalable, and developer-friendly tech stack that prioritizes smooth user interaction, chatbot integration, and reliable deployment.

Frontend

We are using React.js, a powerful JavaScript library for building interactive and responsive user interfaces. React enables us to deliver a seamless user experience, allowing students to easily create, view, and update their schedules in real time. Its component-based architecture also ensures fast development and clean code management.

User Authentication

For handling sign-up, login, and session management securely, we use Supabase Authentication. It provides a simple yet robust authentication system that allows us to manage user identities through secure APIs, ensuring that each student's schedule and preferences remain private and protected.

Backend

Our backend is built with Express.js, a minimalist and flexible Node.js framework. It handles all server-side logic, API routing, and communication between the frontend and various services. Express provides us with a lightweight structure that is easy to maintain and extend. And MongoDB

Chatbot Integration

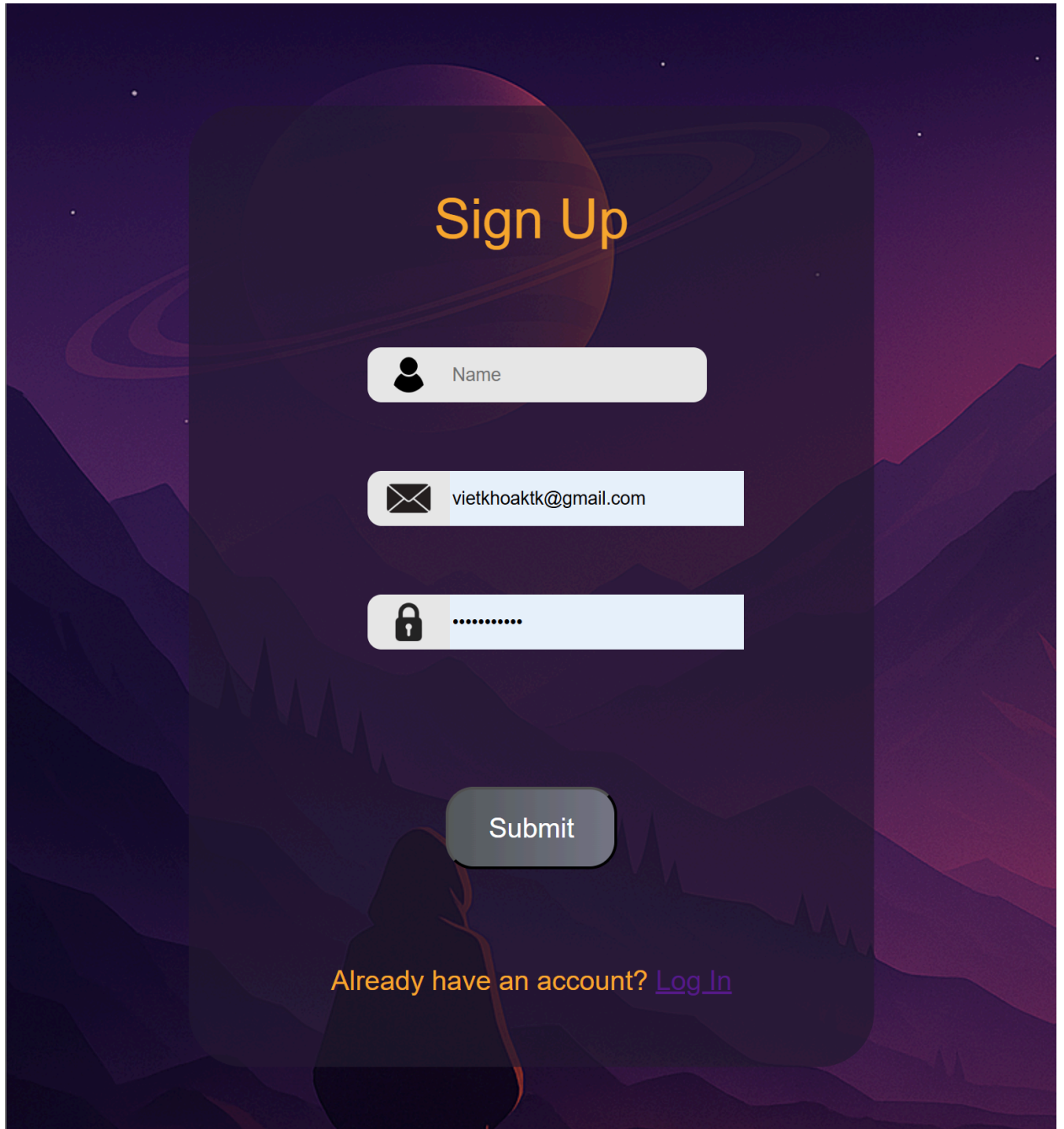
A core feature of ScheduleBoss is its chatbot assistant, designed to help users manage their tasks, navigate the platform, and receive intelligent scheduling tips. This is powered by the Gemini API, a conversational AI model that enhances the interactivity and usability of our platform. The chatbot serves as a friendly companion for time management and productivity support.

Hosting & Deployment

To ensure that our web application is accessible and runs smoothly, we use Render to host both our frontend and backend services. This allows us to deploy updates efficiently and monitor performance in real time. Our codebase is managed using GitHub, enabling seamless collaboration, version control, and continuous integration.

Features Implemented

1. Log in/Sign Up page



This page has been fully developed, with both the frontend user interface and the authentication system thoroughly implemented and tested to ensure a smooth and secure experience for users. To manage user sign-up

and login functionality, we have integrated Supabase, a modern Backend-as-a-Service (BaaS) platform known for its simplicity, scalability, and robust authentication features.

Supabase handles user authentication through secure, token-based email verification. When a new user attempts to register, they are prompted to enter their email address. A verification email is then sent to confirm their identity, ensuring that only genuine and authorized users can proceed. This step adds an important layer of security while maintaining a frictionless user experience. After the email is verified, users can set up their account and gain access to the platform's features.

For returning users, the process is even more seamless, they simply need to sign in using their email and password, and Supabase takes care of the session management behind the scenes. The system also supports persistent login states, allowing users to stay signed in across sessions unless they choose to log out manually.

By using Supabase, we not only minimize the complexity of implementing custom authentication logic, but we also gain access to real-time user management, scalable storage, and robust backend tools that ensure our application remains secure, maintainable, and user-friendly.

By storing the user's session information in **local storage**, the application preserves their login state **across sessions**. As a result, users won't be logged out unless they explicitly choose to do so by clicking the logout button. This provides a smoother and more convenient user experience, eliminating the need for repeated sign-ins while maintaining control over session termination.

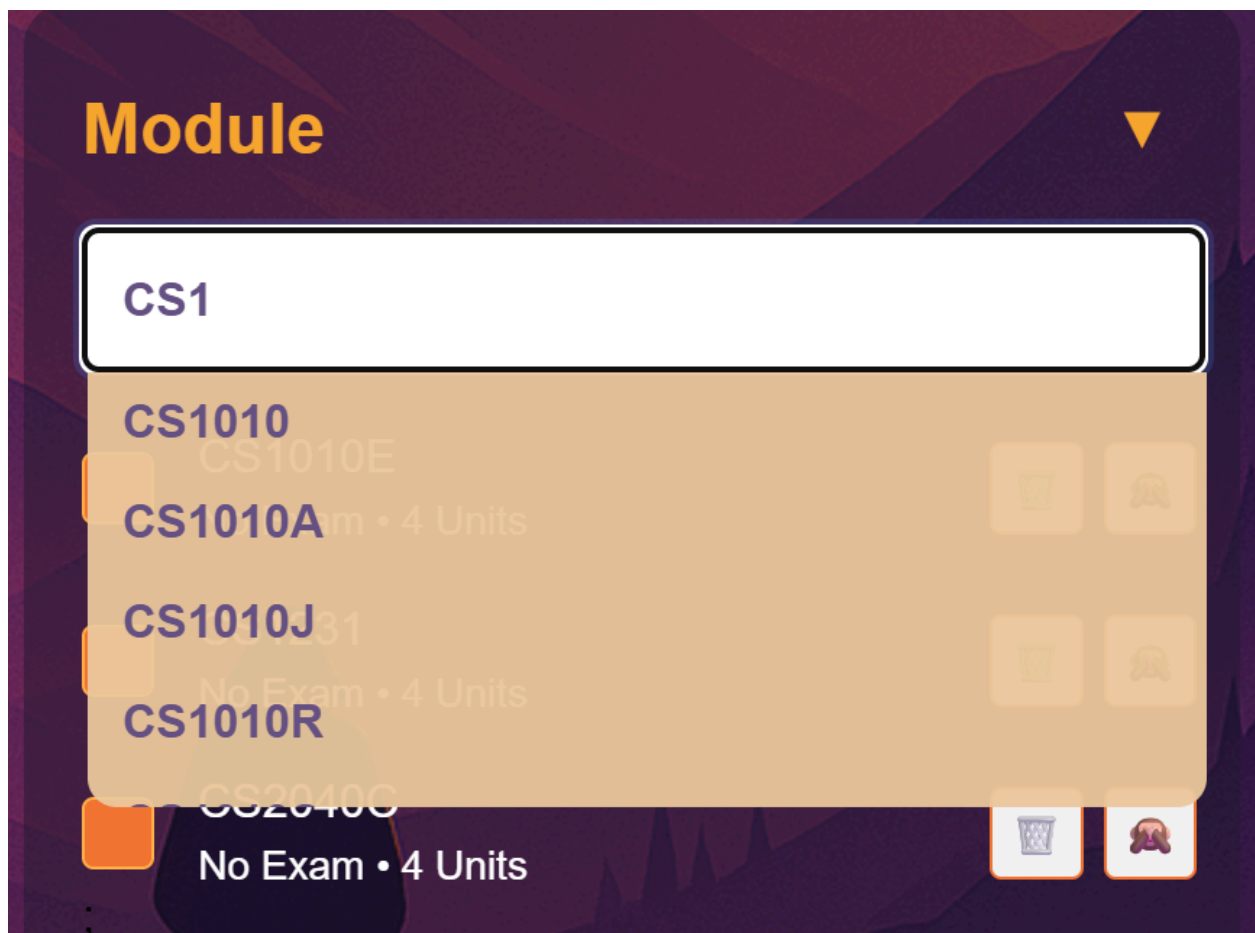
2. Timetable

After successfully logging in, users are seamlessly redirected to the **main dashboard**, which serves as the central interface for managing their academic and personal schedules. This page is designed to be both intuitive and

feature-rich, enabling students to take full control of their time through a combination of module planning and task management

a. Module Management

Users can effortlessly incorporate academic modules into their personalized timetable by utilizing a dynamic and responsive search bar. This feature allows them to search for specific modules by entering their unique codes, for instance, "CS1010E" or "CS2040C", making the process both quick and intuitive. Moreover, a section will display modules whose initial letters match the characters the user is typing in the search bar.



Once a module is selected, the corresponding time slots associated with that module will momentarily flicker or highlight on the timetable. This visual cue helps users quickly identify when the module is scheduled without any confusion. If the user clicks on any of the highlighted time spans, that specific span will be marked as selected, effectively adding it to the user's active timetable view.

This interactive approach enhances user experience by combining search efficiency with clear visual feedback, ensuring the timetable is both accurate and easy to navigate.

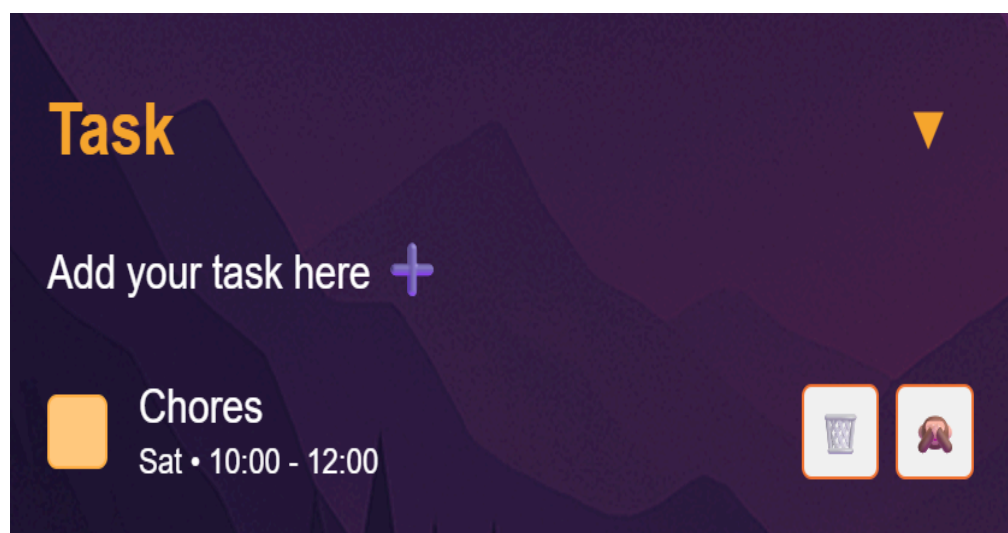
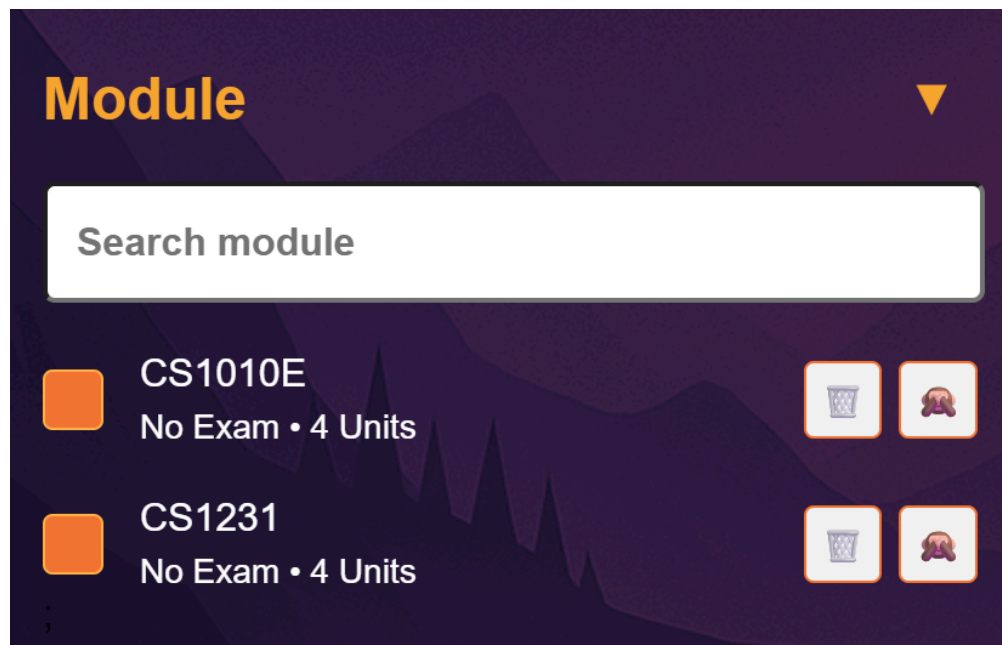
	Mon	Tue	Wed	Thu
08:00				
09:00				
10:00				
11:00				

11:00		
12:00	CS1231	
13:00	CS1231	
14:00		
15:00		
16:00	CS1010E	
17:00	CS1010E	
18:00		

b. Module/ Task/ Span Controls

Each module in the container comes with built-in control options:

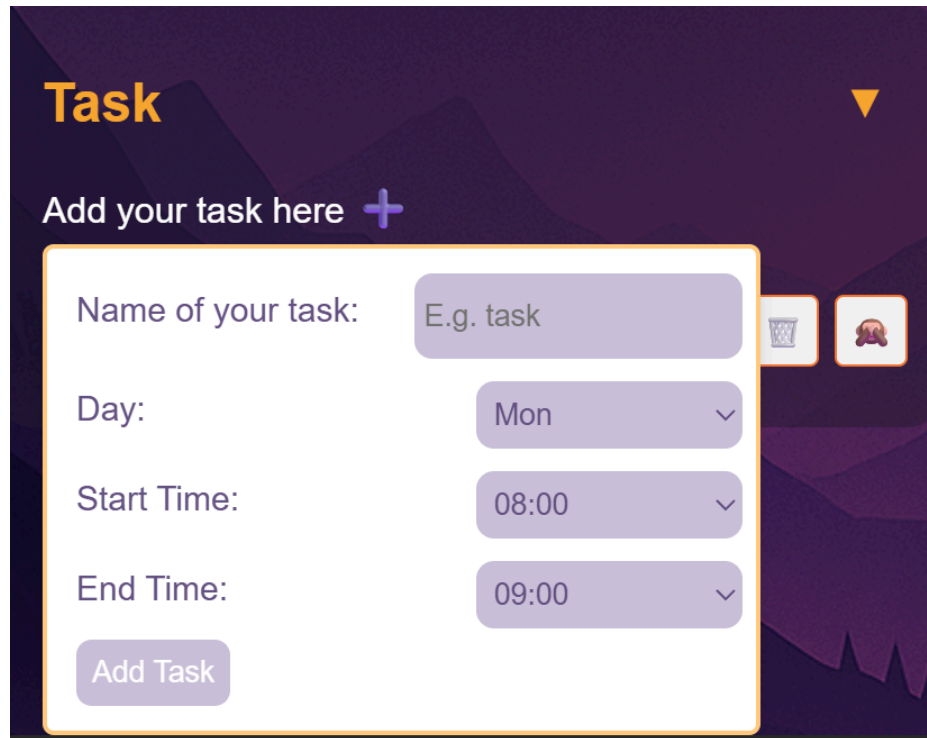
- 🗑️ **Delete** a module by clicking the trash bin icon.
- 🐒 **Hide or unhide** individual modules using the monkey icon—perfect for decluttering the view.
- Use the two square icons located in the top-right corner of the timetable to **toggle the visibility** of **all modules** or **personal tasks**, offering a flexible and customizable interface depending on the user's focus or preference.



c. Task/ Section span addition

Users have the flexibility to add their own personal tasks directly to the timetable, allowing for a more comprehensive and customized scheduling experience. This can be done by filling in the designated input fields, as illustrated in the accompanying image. These fields typically prompt users to enter relevant details such as the task name, start and end time, date, and any additional notes or descriptions.

Users can seamlessly integrate non-academic responsibilities, such as study sessions, errands, or personal appointments, into their weekly schedule. This feature ensures that the timetable not only reflects academic commitments but also supports effective time management for all aspects of a user's daily life.



The screenshot shows a mobile application interface for adding a task. The background is a dark purple gradient with a faint mountain silhouette. At the top left, the word "Task" is written in a bold, orange font. To its right is a small orange downward-pointing triangle. Below "Task" is the text "Add your task here" followed by a purple plus icon. A white modal form with a thin orange border is centered on the screen. Inside the form, there are four input fields, each with a label on the left and a light purple rounded rectangle on the right. The first field is labeled "Name of your task:" and contains the placeholder text "E.g. task". To the right of this field are two small icons: a trash can and a person with a backpack. The second field is labeled "Day:" and contains the text "Mon" with a small downward arrow on the right. The third field is labeled "Start Time:" and contains the text "08:00" with a small downward arrow on the right. The fourth field is labeled "End Time:" and contains the text "09:00" with a small downward arrow on the right. At the bottom left of the form is a light purple rounded rectangle button with the text "Add Task".

Task

Add your task here +

Name of your task: E.g. task

Day: Mon

Start Time: 08:00

End Time: 09:00

Add Task

Sports

✕ ▼

Add your sports here +

Name of your sports:

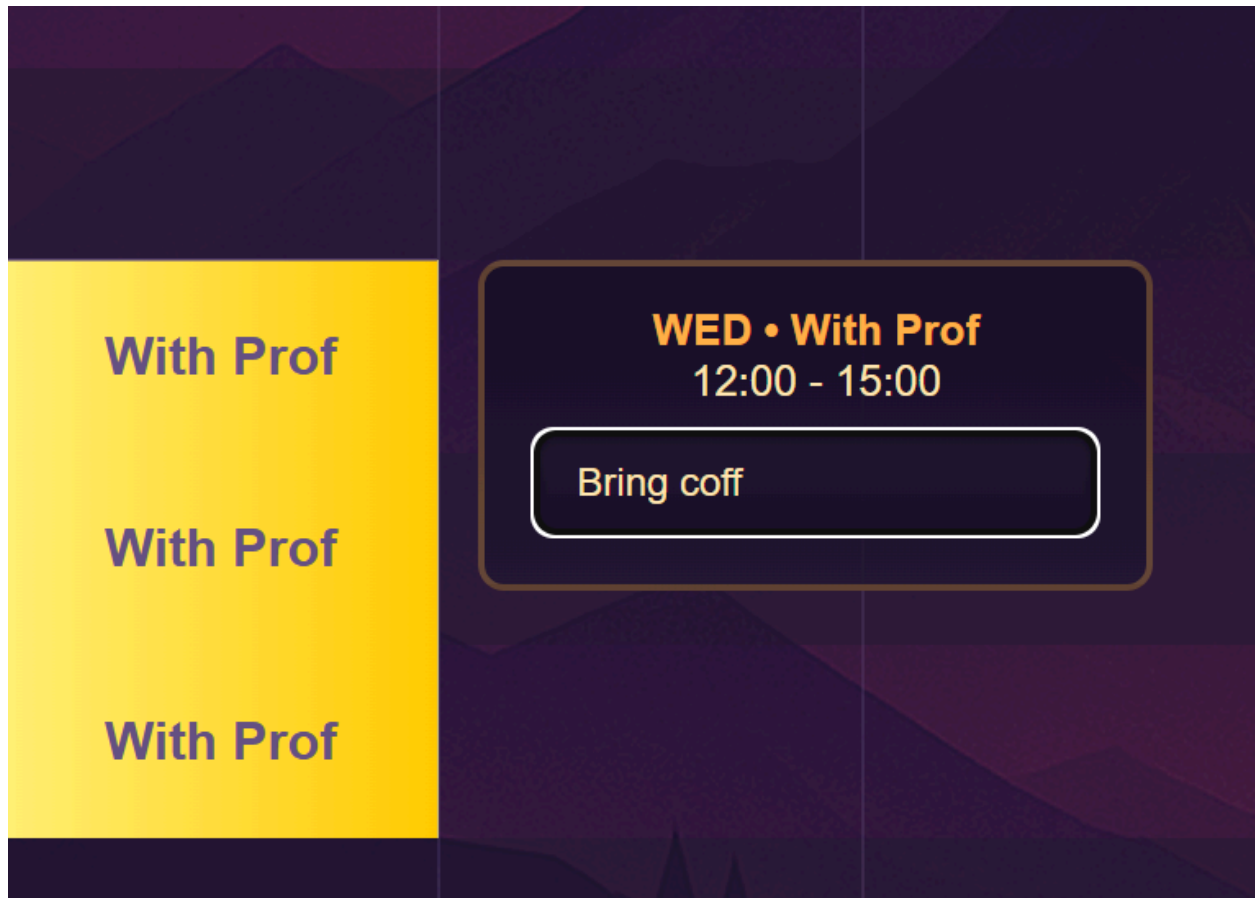
Day:

Start Time:

End Time:

Add Sports

Furthermore, users have the ability to enrich their scheduled spans with additional information. By simply hovering over a span, a detailed view appears, allowing users to conveniently input or update relevant details associated with that specific time block.



d. “My Calendar” Widget

This widget offers users an intuitive way to manage and filter their schedule visually. It features color-coded boxes for each main category, **Module**, **Task**, **Meeting**, and **Sports** (custom sections) which directly correspond to the colored spans shown on the timetable. These color indicators serve as a legend and are consistently applied across the interface for easy recognition.

By clicking on any of the colored boxes in the widget, users can toggle the visibility of that entire section on the timetable. For instance, clicking the yellow box will show or hide all personal tasks, while clicking the red-orange box will control the visibility of all meetings. This allows users to focus on specific aspects of their schedule based on their current priorities.

In addition to controlling visibility, the color of each span on the timetable matches the respective box color from the widget. This ensures that users can

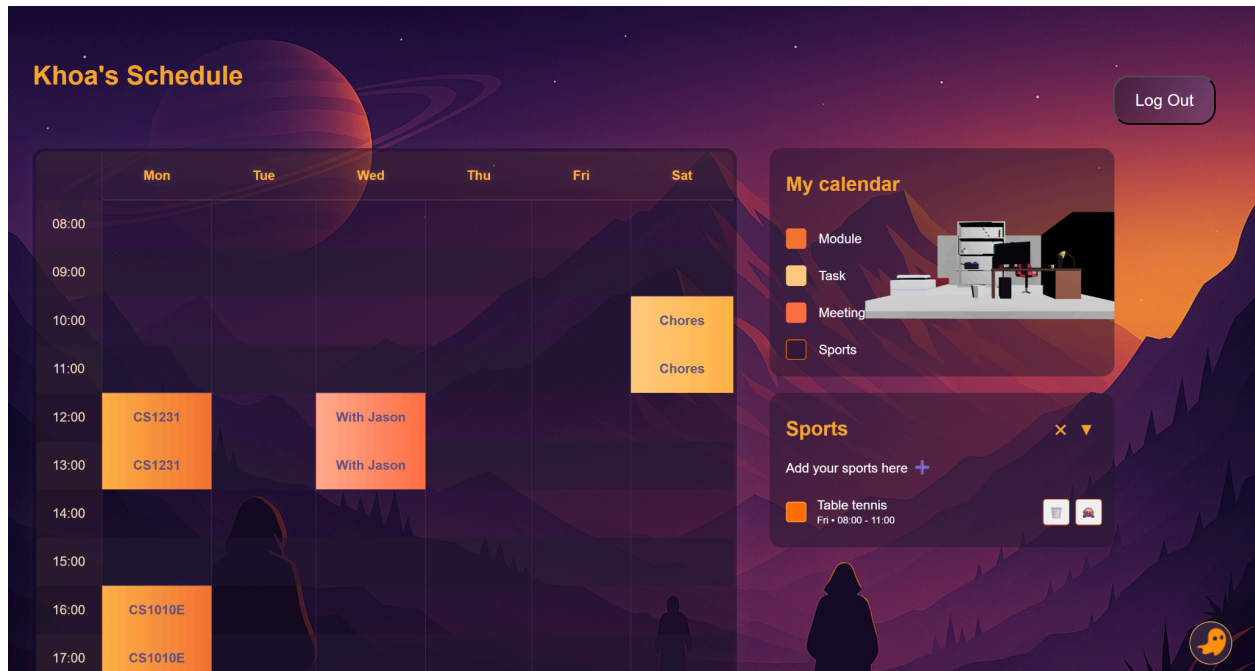
immediately identify the category of any given time block at a glance, resulting in a clean, organized, and user-friendly calendar experience.



e. Backend

The backend of our application is designed to seamlessly support the personalized scheduling experience offered on the frontend. For every user, the system stores a structured set of calendar data that includes distinct sections, such as **Module**, **Task**, **Meeting**, and **Sports**, each defined by its own color, visibility state, and a collection of time spans. These spans represent the individual events or activities scheduled within that category, complete with details such as title, day, and time range.

Each section's assigned color is saved in the database and consistently rendered on the timetable to visually distinguish it from others. In addition, the visibility state of each section is also tracked, allowing the frontend to show or hide entire categories based on user preference with a single click. By preserving this level of detail on the backend, the system ensures that users' timetables remain synchronized, customizable, and visually coherent across sessions and devices.



f. Database

Our MongoDB database is carefully structured to support a flexible and personalized scheduling system for each user. At its core, each user document includes three main components: **sections**, **spans**, and **colors**. The section field stores an array of maps, where each map defines the visibility status (true or false) of different calendar categories, such as Module, Task, Meeting, or Sports. This enables the system to track which sections the user wants visible on the timetable at any given time.

The spans array holds detailed scheduling entries. Each span contains information about the **day**, **start and end times**, the **name** of the activity, its **type** (e.g., Task or Module), and a visibility map that allows for fine-grained control over its display status across different views or contexts. This flexible span structure supports multiple event types while maintaining consistency in how data is accessed and displayed.

Finally, the colors field is an array of maps that associate each section type with its designated color codes, separately defined for the **box** (used in the calendar legend) and the **span** (used on the timetable itself). This ensures visual consistency across the UI and supports easy customization. Overall, the schema

is designed to align closely with the frontend behavior, allowing for efficient data retrieval, real-time updates, and a seamless, user-friendly calendar experience.

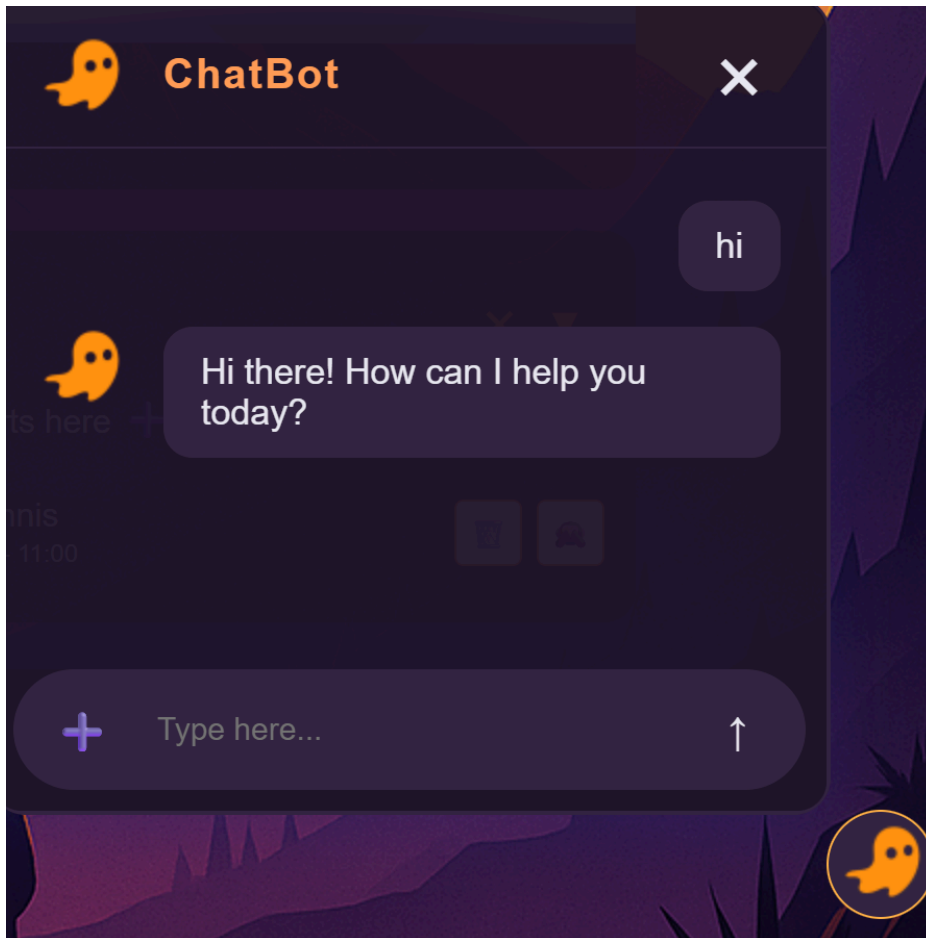
```
1  const mongoose = require('mongoose');
2
3  const spanSchema = new mongoose.Schema({
4    day: { type: String, required: true },
5    start: { type: String, required: true },
6    end: { type: String, required: true },
7    name: { type: String, required: true },
8    type: { type: String, required: true },
9    visibility: { type: Map, of: Boolean, default: {} },
10  });
11
12  const colorSchema = new mongoose.Schema({
13    box: String,
14    span: String
15  }, { _id: false });
16
17  const userSchema = new mongoose.Schema({
18    userEmail: { type: String, required: true, unique: true },
19    sections: [{type: Map, of: Boolean}],
20    spans: [spanSchema],
21    colors: [{type: Map, of: colorSchema}]
22  });
23
24  module.exports = mongoose.model('User', userSchema);
```

Chatbot

- a. Normal messages with persistent message history

The **Chatbot** feature enhances user interaction by providing a conversational interface that supports normal, free-form messages. Users can type questions and give commands. To ensure a smooth and continuous user experience, all message exchanges are stored persistently in the database. This means that each time a user returns to the chatbot, their previous conversation history is automatically loaded, allowing them to pick up where they left off without losing context.

Additionally, users have the option to start a **"New Chat"** (By clicking the plus button), which clears the current conversation thread and begins a fresh session. This is especially useful when users want to switch topics or reset the conversation without distractions from previous messages, providing both continuity and flexibility in how they engage with the chatbot.



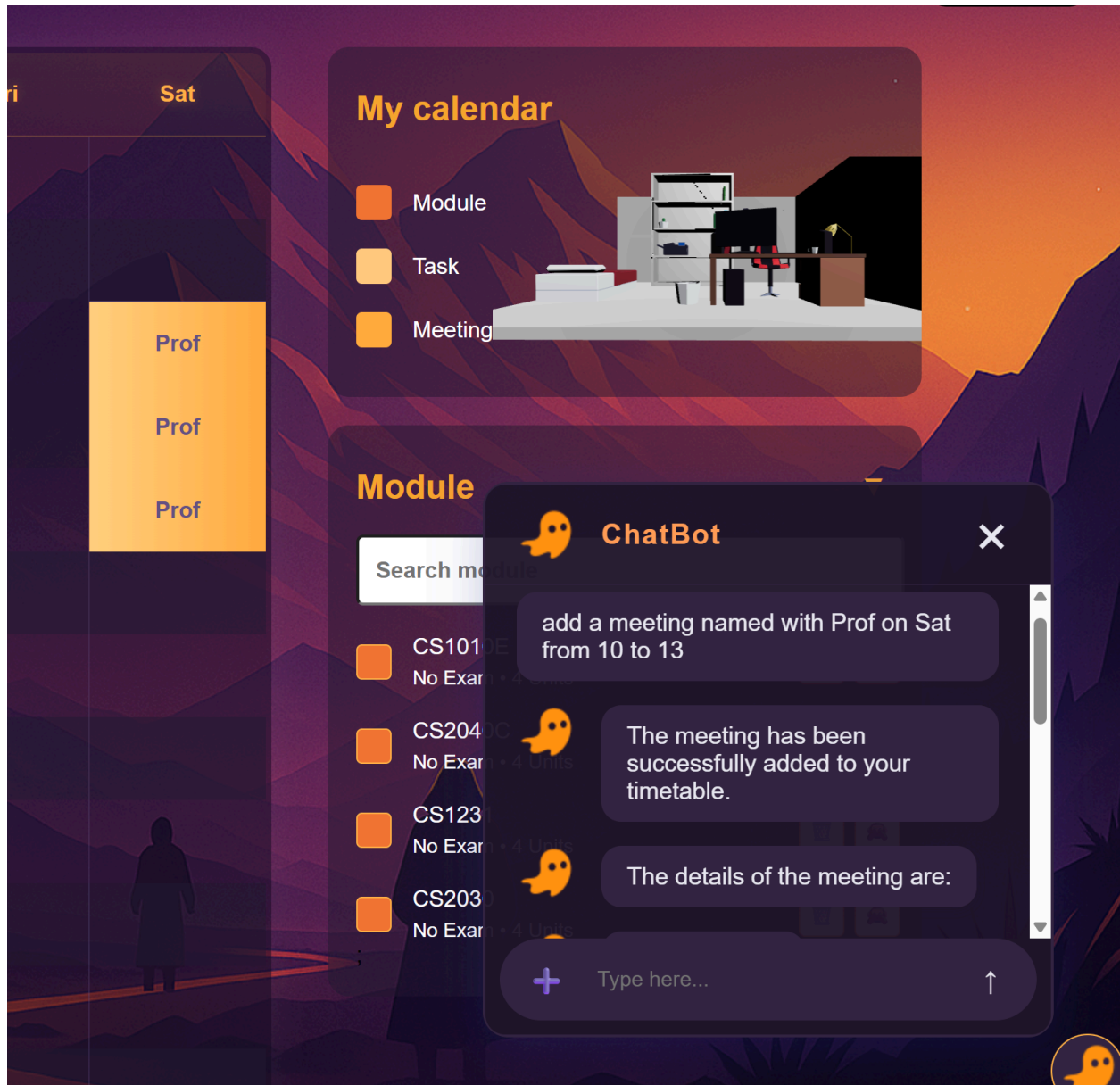
b. Database

The backend database for the **Chatbot** is structured using MongoDB to support persistent, organized message histories for every user. Each user document stores the user's email, a chatActive flag that indicates whether the chatbot session is currently open, and a messages map that holds arrays of conversation entries. This map-based design allows messages to be grouped by key, such as by date, enabling more structured storage and efficient retrieval of chat history.

Each message within the arrays follows a consistent schema, containing a sender field (either 'user' or 'bot') and the corresponding message text. This clear distinction between user and bot messages ensures the conversation flow is accurately preserved and displayed in the correct order. By maintaining persistent message records in this way, the chatbot can deliver a more contextual and intelligent user experience, allowing users to resume conversations, reference past interactions, or continue multi-step tasks without losing valuable context.

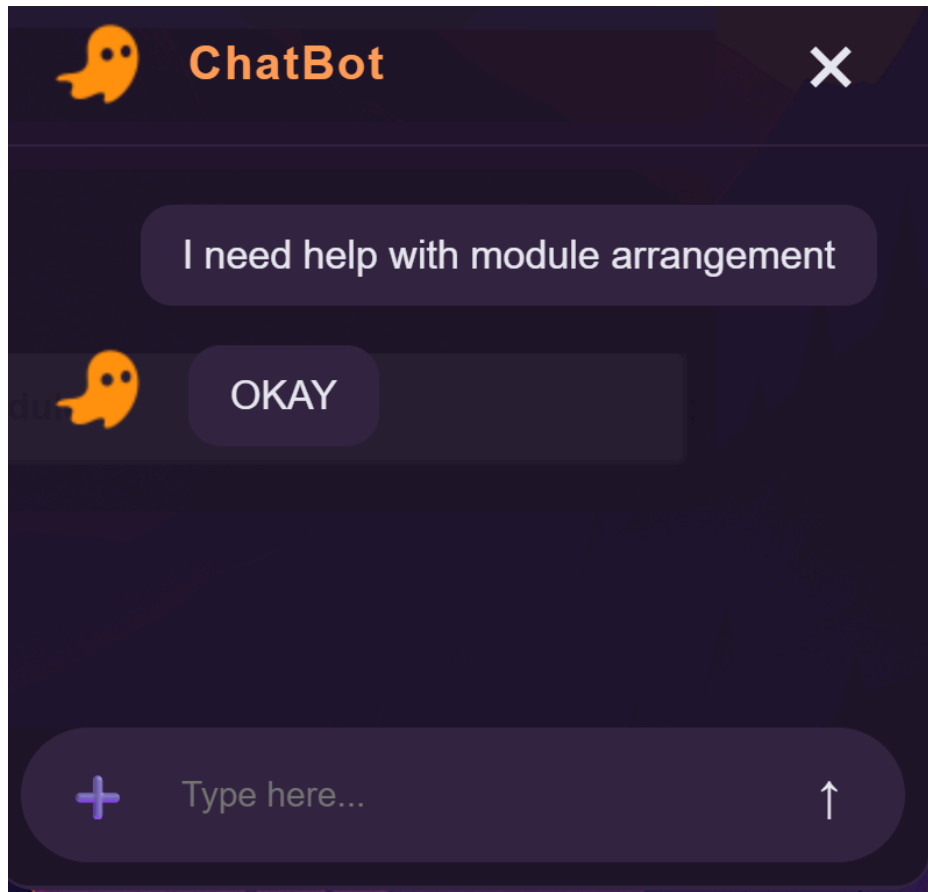
```
1  const mongoose = require('mongoose');
2  const messageSchema = new mongoose.Schema({
3    sender: { type: String, required: true, enum: ['user', 'bot'] },
4    text: { type: String, required: true },
5  },
6  )
7
8
9  const userSchema = new mongoose.Schema({
10    userEmail: { type: String, required: true, unique: true },
11    chatActive: { type: Boolean },
12    messages: {
13      type: Map,
14      of: [messageSchema]
15    }
16  })
17
18  module.exports = mongoose.model('User_chat', userSchema)
```

Add module/ task/ ... spans



Modules Arrangement

Module arrangement is a time-consuming challenge that frustrates most university students. NUS Schedule Boss eliminates this hassle through an intelligent, conversational approach. Users simply type their scheduling request into the chat interface, such as "Help me arrange my modules for this semester" .



Upon receiving the request, a user-friendly form appears where students input their desired modules and specify whether they want to avoid conflicts with

personal commitments like work or extracurricular activities.

Module Scheduler

×

Modules to Schedule (comma-separated)

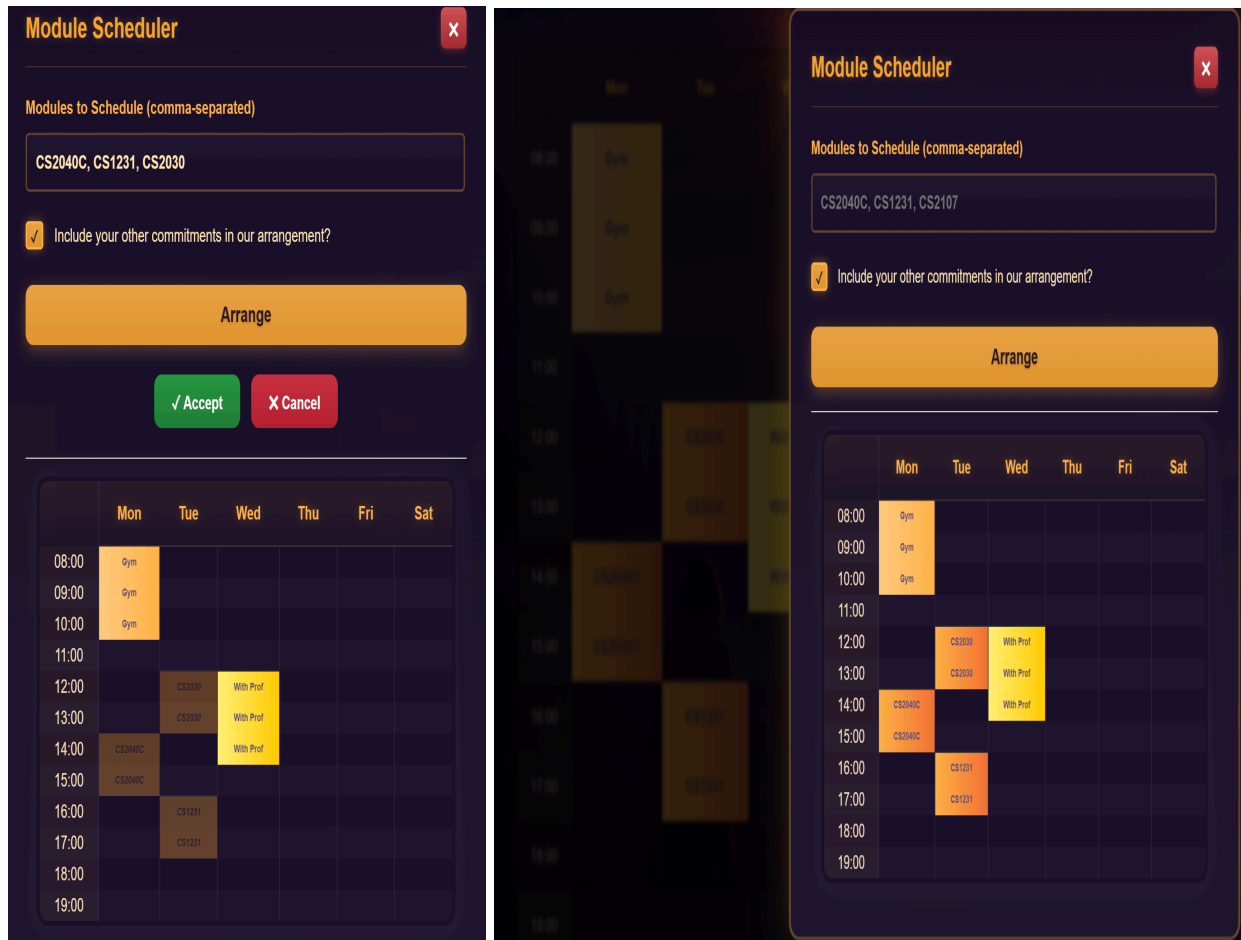
CS2040C, CS1231, CS2107

☐ Include your other commitments in our arrangement?

Arrange

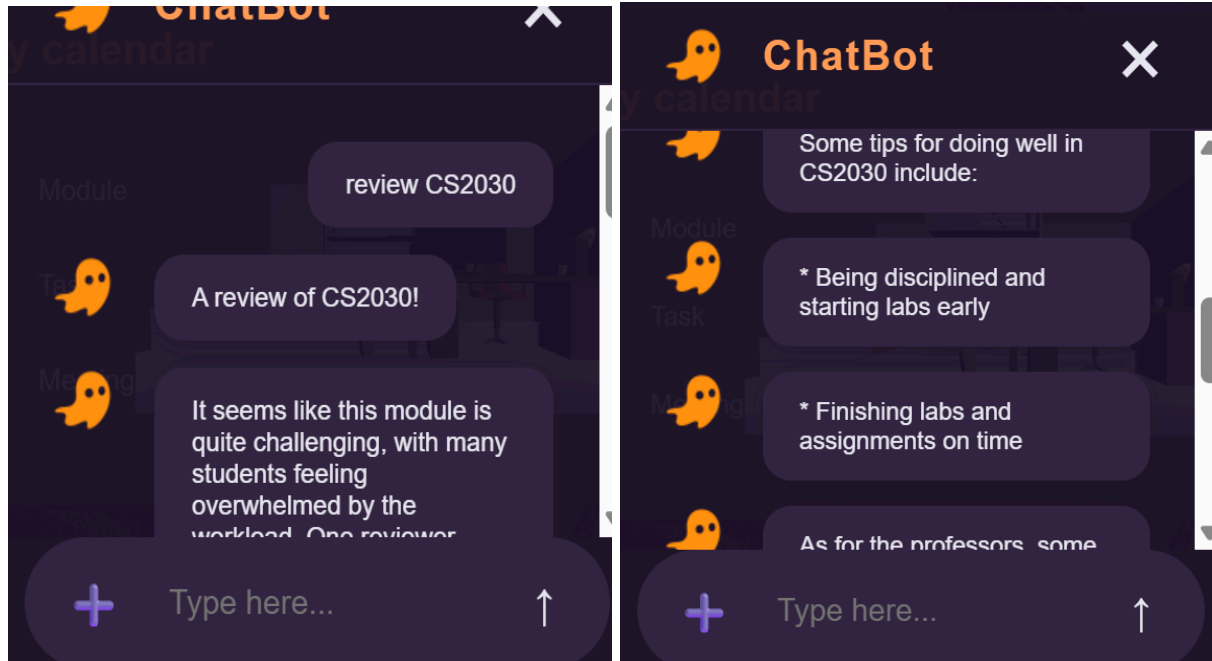
	Mon	Tue	Wed	Thu	Fri	Sat
08:00	Gym					
09:00	Gym					
10:00	Gym					
11:00						
12:00			With Prof			
13:00			With Prof			
14:00			With Prof			
15:00						
16:00						
17:00						
18:00						
19:00						

After clicking "Arrange," the system intelligently generates an optimized timetable, with the arranged modules appearing with a subtle flickering animation on the schedule grid. Users can then review the proposed arrangement and either "Accept" to confirm the slots or "Cancel" to try different options.



Module Reviews

This feature allows users to request reviews for specific modules by entering commands such as "Review CS2030 for me" or "Show me reviews for MA1521", making the process both conversational and intuitive. The chatbot automatically recognizes module codes within the user's message and initiates the review gathering process. Once a review request is submitted, the chatbot seamlessly scrapes relevant discussions from the NUS subreddit, aggregating authentic student experiences and opinions about the requested module. The system compiles these reviews into a structured format, presenting key insights such as difficulty ratings, workload expectations, teaching quality feedback, and practical advice from students who have completed the module.



##Disclaimer: *This feature involves searching and scraping information from the internet, which may take some time, up to 2 minutes in some cases. We appreciate your patience and apologize for any inconvenience. 🙏*

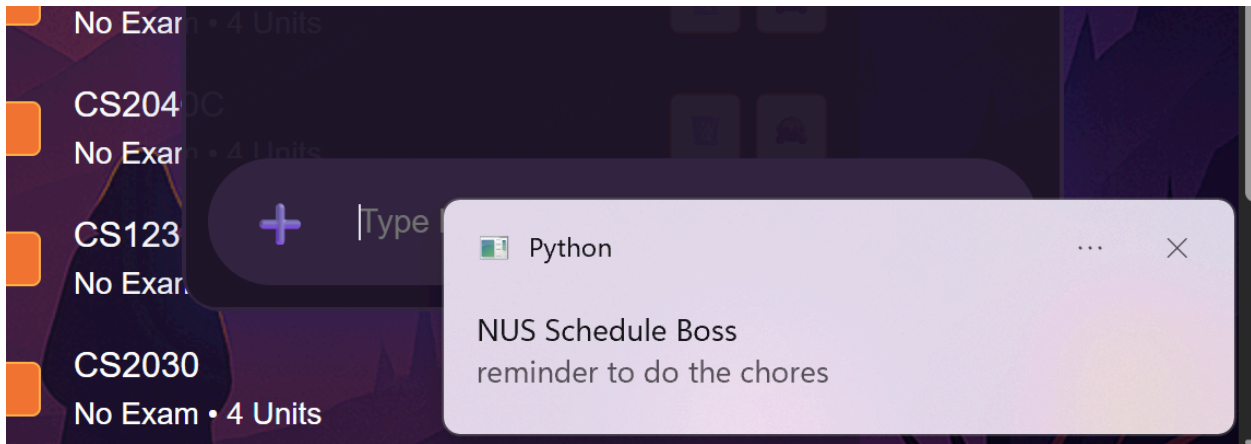
Pop up notification

The **Pop-up Notification** feature enables users to set immediate reminders through natural conversation. Users can simply type commands like "remind me to go to class in 5 mins" or "remind me to call John in 30 minutes," and the chatbot intelligently parses the time delay to create a scheduled notification.

When a reminder is set, the chatbot confirms the details and uses a delay mechanism to trigger the notification after the specified time period. The system calculates the exact delay duration from the user's input and starts a countdown timer in the background. Once the timer expires, a pop-up notification appears on the user's screen with the reminder message, even if they've navigated away from the chatbot interface.

The feature supports relative time formats such as "in 5 minutes," "in 2 hours," or "in 30 seconds," making it perfect for short-term reminders and immediate scheduling needs. The notifications appear as styled pop-ups with the reminder

text and a dismiss button, transforming the chatbot into a helpful assistant that keeps users on track with their immediate tasks and appointments.



Software Integration Workflows

Software Testing

Manual Unit Tests

After building a new feature, we conduct a structured series of manual tests to ensure each core function of the app works as expected on its own. These tests follow a consistent, step-by-step process designed to catch bugs or irregular behavior early. This helps us resolve issues efficiently before merging new features into the main website experience.

Target Features	User Actions	Expected Outcome
Add modules	1. The user enters the module code in the search bar. 2. The user clicks on one of the available time slots for the	1. The module code is searched in the database, and an error is shown if the module is not

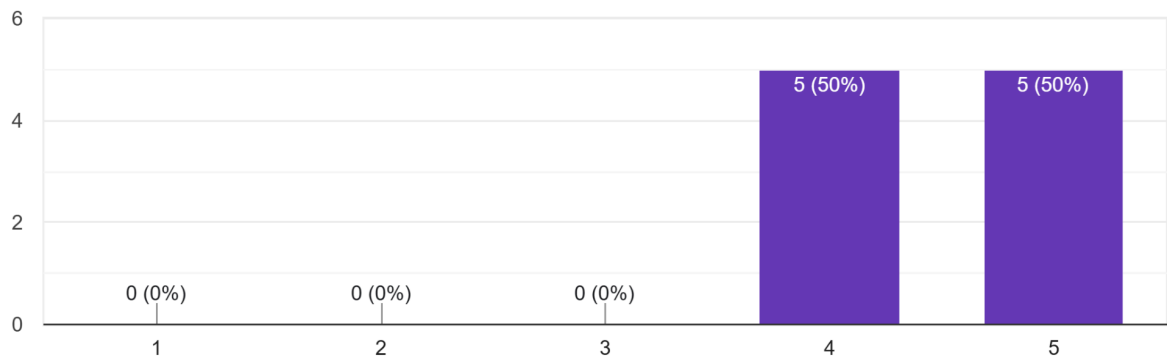
	module, which flicker on the timetable widget.	found. 2. The modules span will be selected on the timetable
-Create new section -Add section spans to timetable	1. The user clicks on the dropdown menu and types the name of the new section. 2. In the newly created section, the user clicks on the '+' button to input the required information and add it to the timetable. 3. User hovers over the span in the timetable widget (not yet implemented).	1. The section automatically switches to the newly added one upon pressing Enter. The section will appear in the "My Calendar" widget, with a colored box in front of it. The color of the box and the section spans will match and be randomly assigned upon creation. 2. The span will be added to the timetable, and a list of spans for that section will be displayed. 3. When a span is hovered over, its details will be shown (Ongoing).
Visibility toggling/ Deletion	1. The user clicks the "monkey" or "trash" button in the section widget. 2. User clicks on the color box of the section in the 'My calendar' widget	1. Corresponding spans will be hidden/unhidden when the "monkey" icon is clicked, and deleted when the "trash" icon is clicked. 2. Entire sections can be hidden or shown.
Chatbot - [Arrange and add modules]	1. The user asks the chatbot to help them arrange modules 2. The user types a list of desired modules and decide whether they want to avoid conflicts with their other commitments 3. User clicks arrange and decide to accept or cancel the arrangement	1. A user-friendly form will appear. 2. Modules' timeslots will be calculated and arranged slots will be flickering on the timetable and be added if clicked 'Accept'.

Chatbot - [to add spans]	1. the user types in the information of the span they want to add to the timetable, including: type, name, day, start, and end (Example: add a task named Chores to the timetable on Sat from 9 to 10)	1. A span is selected and highlighted on the timetable 2. if the type of span is not included in current sections, a new section will be created as well.
Chatbot - [to give reviews]	1. User asks for opinion of a module	1. The chatbot will search on NUSMods and Reddits and return a review

User Testing

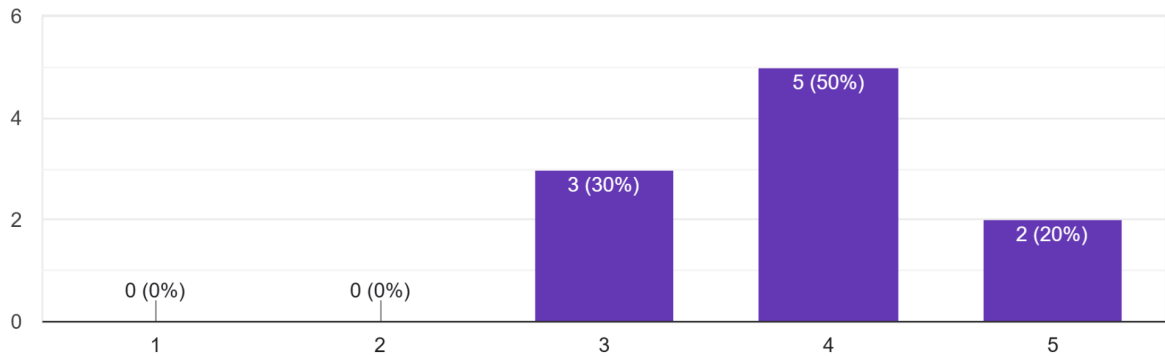
How well do you understand the website's functions?

10 responses



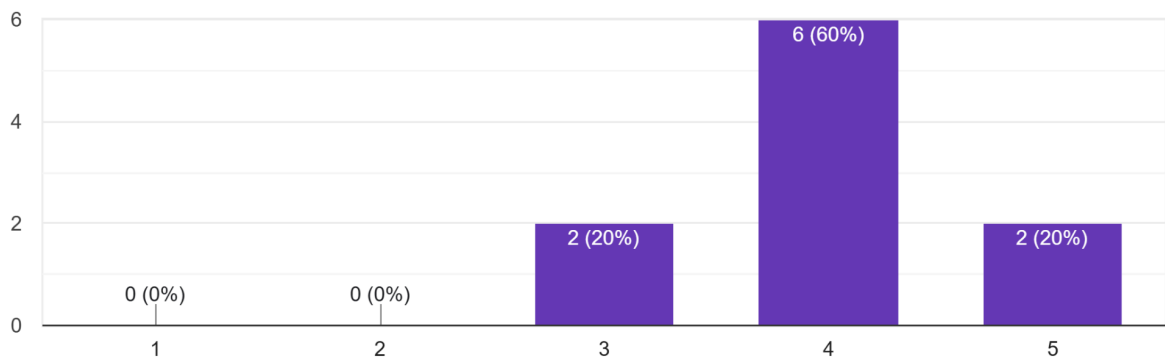
Does the website help you schedule your courses easily?

10 responses



Would you use this website in real life if it were fully functional?

10 responses



Testing methodology:

We hosted our website and generated a link for users to try it out. We conducted testing with users (our friends) from various regions in Vietnam and with diverse majors and backgrounds. Each participant was asked to spend around 10 minutes exploring the features and providing feedback afterward.

Areas for improvement:

- The chatbot should automatically scroll to the latest message.
- Sections should be visible by default.

- New sections should be automatically added to the “My Calendar” widget when entered.
- The chatbot should display past messages when users log in again.
- Span colors should correspond to the color boxes of their respective sections.
- Add an option to start a new chat.
- Allow section deletion.

Bugs Identified:

- No error is shown when adding a span to an already-filled time slot on the timetable widget.
- No error is shown when trying to add a non-existent module.
- The chatbot can add modules with non-existent time slots to the timetable.

End to end (E2E) Testing:

⇒ Specs

✓ 26 ✗ 1 ↺ --

⌵ ↺

timetable.cy.js 00:34

▼ Timetable Application

▼ Timetable Grid

✓ should display timetable structure correctly

✓ should show correct days of the week

✓ should display time slots in first column

✓ should handle cell clicks for span selection

✓ should display selected spans with correct styling

▼ Section Management

✓ should display section dropdown with correct options

✓ should switch between sections

✓ should create custom sections

✓ should delete custom sections

✓ should not show delete button for default sections

▼ Module Management

✓ should display module search input

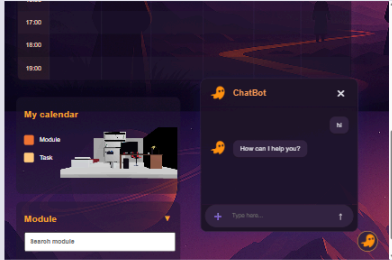
✓ should add new modules

✓ should prevent duplicate modules

✓ should delete modules

✓ should toggle module visibility

🔄 h... 1000x660 22%



▼ Task Management

- ✓ should display task section header
- ✓ should show task creation form
- ✓ should add new tasks
- ✓ should delete tasks
- ✓ should toggle task visibility

▼ Calendar Widget

- ✓ should display calendar section
- ✓ should show calendar items for different sections
- ✗ section visibility should be toggled for "Module"
- ✓ should display color boxes with correct styling



▼ Chat Integration

- ✓ should display chat toggle button
- ✓ should toggle chat visibility
- ✓ should load chat messages from API

To ensure key functionalities of our Timetable Application remain efficient/ robust as we expand features and iterate on the UI, we've implemented a comprehensive suite of end to end (E2E) tests using Cypress. These automated tests simulate real user behavior across multiple critical flows, allowing us to identify bugs early and ensure seamless integration between the frontend and backend.

One of the most critical flows tested is **section and timetable management**, where users can create custom sections, select time spans, and switch between views. Our E2E tests validate that new sections can be added and deleted, custom selections are preserved, and the timetable structure updates accordingly. We also simulate span selection by clicking on time cells, ensuring the selected spans are styled and saved properly.

Another core flow under test is **module management**. Here, we simulate user interactions such as searching for modules, adding them via input, preventing duplicates, and toggling their visibility. These tests also validate that deleted modules are removed from the UI immediately and no stale data remains across sections.

In the **calendar widget**, we ensure that calendar items reflect each section accurately and respond correctly when a user toggles visibility. For instance, a failed test case highlights an issue in toggling section visibility for "Module", helping us identify and isolate logic bugs early.

The **chat integration** is also tested end-to-end. Tests verify that the chat panel opens and closes correctly and that the messages can load from the backend API.

Testing methodology:

We generally followed the following steps to automate **end-to-end testing** using **Cypress**, which allows us to simulate real user interactions across the full stack (UI, frontend logic, and backend APIs). These tests ensure our key flows work in an environment that mirrors actual usage.

1. Application Bootstrapping and Mocking API Responses

- Before each test, we mock key backend responses using `cy.intercept()` for API endpoints such as:
 - `/api/timetable_data/*` for timetable and section data,
 - `/api/chatbot/*` for chat history and status,
 - `/api/timetable/*` and `/api/chatbot/*` for saving operations.
- We use a custom command `cy.loginWithToken()` to simulate a logged-in session by injecting a token into `sessionStorage`.

2. Page Visit and DOM Readiness

- After setting up authentication, we use `cy.visit('/timetable')` to navigate to the main application.
- We ensure all key UI components (e.g., the timetable grid, section dropdown, calendar, chat toggle) are visible using assertions like `cy.get(...).should('be.visible')`.

3. Simulating User Interaction

- Cypress commands simulate real user behavior:
 - `cy.get(...).click()` to simulate span selection, toggling buttons, opening dropdowns, etc.
 - `cy.get(...).type()` to simulate input such as module or section names.
 - `cy.get(...).select()` to pick time ranges for tasks.
- We also created **custom Cypress commands** in `commands.js` (e.g., `cy.addModule()`, `cy.addTask()`, `cy.toggleChat()`) to abstract repetitive user flows for clarity and reusability.

4. API Integration and Real-Time Assertion

- These tests validate full-stack behavior, such as whether:
 - the module/task actually appears after being created,
 - the correct number of items is displayed after deletion (`cy.shouldHaveModuleCount()`),
 - visibility toggles update both UI state and calendar views.
- We use `cy.wait('@getTimetableData')` or similar aliases to synchronize with mocked API responses when necessary.

5. Edge Case and Error Handling

- To test application robustness, we:
 - Simulate duplicate entries and ensure they are blocked (`cy.shouldHaveModuleCount(1)`),
 - Test visibility toggling to switch between 🧑 and 👁 icons,
 - Use commands like `cy.mockNetworkError()` or `cy.mockAPIError()` to simulate backend/server issues and observe fallback UI behavior.

6. Viewport and Responsive Testing

- We verify layout responsiveness using:

- `cy.viewport('iphone-6')` for mobile,
`cy.viewport('ipad-2')` for tablets,
- and large resolutions for desktop,
followed by assertions to check for layout visibility (`.timetable`,
`.calendar`, etc.).

7. Assertions and DOM Queries

- We used Cypress's assertion methods for reliable state checking:
 - `.should('contain', ...)`, `.should('not.exist')`, `.should('have.class', ...)`
 - Scoped checks within components (e.g., `.within(() => {...})`)
- We also used aliasing (`.as()`) for chained assertions on toggled states and reused components.

User Flow

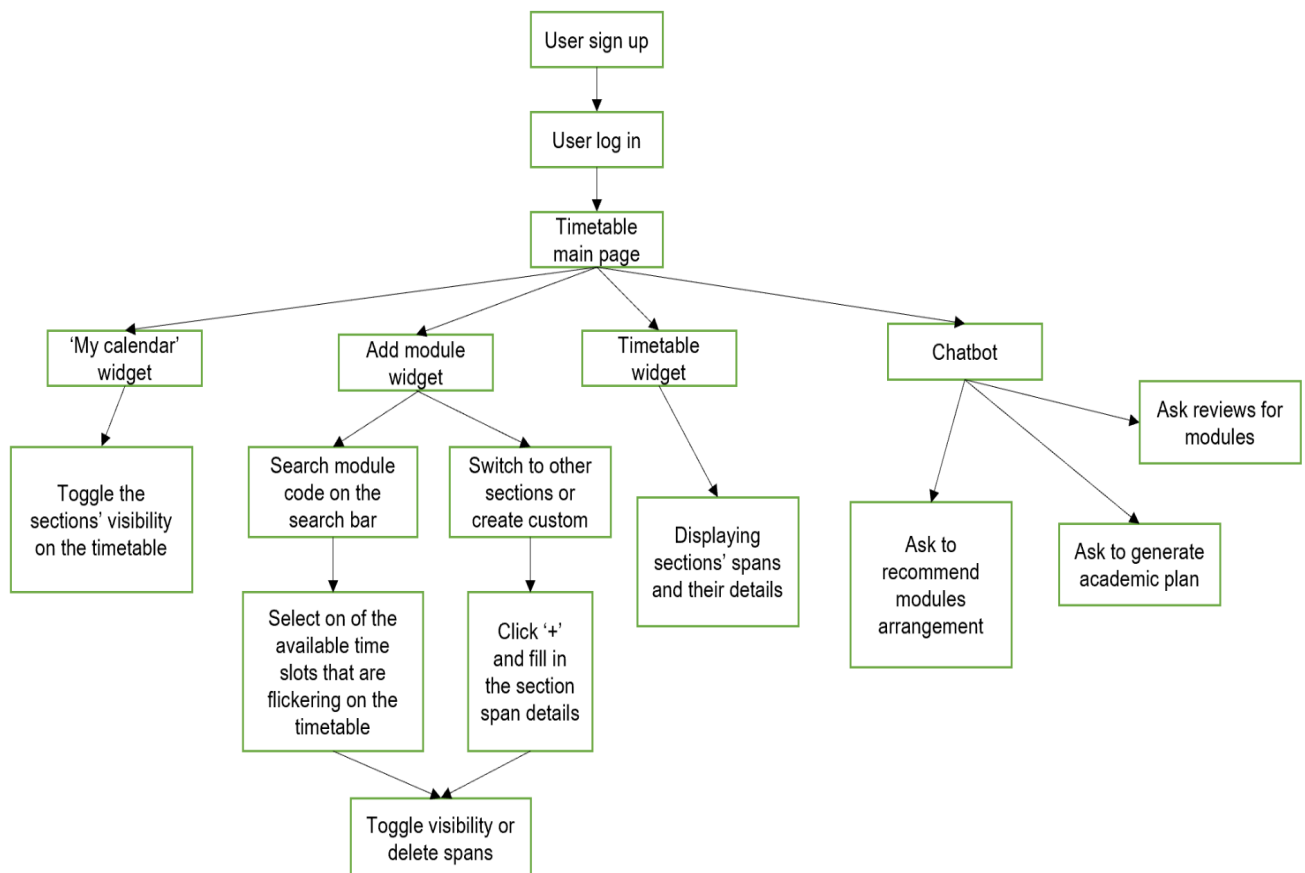


Fig: User Flow Diagram

Software Engineering Principles

Below are some SWE Principles we have tried to incorporate in our code:

1. Separation of Concerns Principle (SoC)

We structured our frontend following the Separation of Concerns principle by clearly dividing responsibilities across components, hooks, utility files, and CSS.

- Timetable.jsx is responsible only for rendering the overall timetable layout and delegating specific functionality to subcomponents.
- ModuleSection.jsx, GeneralSection.jsx, and Chat.jsx each encapsulate a distinct feature: module selection, custom sections, and the chatbot, respectively.
- UI-related state logic (e.g., dropdown state, hover effects, selection highlights) is centralized in a custom hook useUI.js. This prevents cluttering components with state logic and improves reusability and testability.
- Styling is isolated in Timetable.css, keeping presentation concerns separate from functional logic.

This structure made the project easier to navigate, debug, and extend, for instance, we were able to add a new CustomSection.jsx with minimal changes elsewhere, thanks to the clear separation.

```

    </div>
    {activeSection.toLowerCase() === 'module' && ( ...
    )}
    {activeSection.toLowerCase() !== 'module' && ( ...
    )}
  </div>
</div>
</div>

<div className={`chat-container ${showChat ? ' chat-container-active' : ''}`}>
  <Chat setModules={setModules} ...
    setMessages={setMessages}
  />
</div>
```

2. Open/ Closed Principle (OCP)

We designed key features to be **open for extension, but closed for modification**.

For example, users can dynamically create new sections by typing their name into an input field.

These new sections are automatically integrated into:

- The dropdown for selection,
- The calendar UI (with its own color),
- Span rendering logic for scheduling tasks.

This works **without requiring any manual changes** to existing logic or conditionals. The system recognizes the new section, assigns it a color, and renders it just like a predefined one, demonstrating extensibility by design.

```
<input type="text" className="dropdown-input" placeholder="Others.." value={newSectionName}
onChange={(e) => setNewSectionName(e.target.value)}
onKeyDown={(e) => {
  if (e.key === 'Enter' && newSectionName.trim()) {
    addCustomSection();
    setActiveSection(newSectionName);
    setDropdownOpen(false);
  } else if (e.key === 'Escape') {
    setNewSectionName('');
  }
}} autoFocus/>
```

And Flexible section rendering

```
<div className="calendar-check">
  {allSections.map((section) => {
    const sectionColor = sectionColors?.[section]?.box || '#ffb347';
    return (
      <div
        key={section}
        className={`calendar-item ${sectionVisibility[section] ? 'active' : ''}`}
      >
        <div
          className="color-box"
          style={{
            backgroundColor: sectionVisibility[section] ? sectionColor : `rgba(35, 25, 50, 0.5)`,
            border: `1px solid ${sectionColor}`
          }}
          onClick={() => {
            toggleSectionVisibility(section);
          }}
        ></div>
        <div className="item">{section}</div>
      </div>
    );
  })}
</div>
```

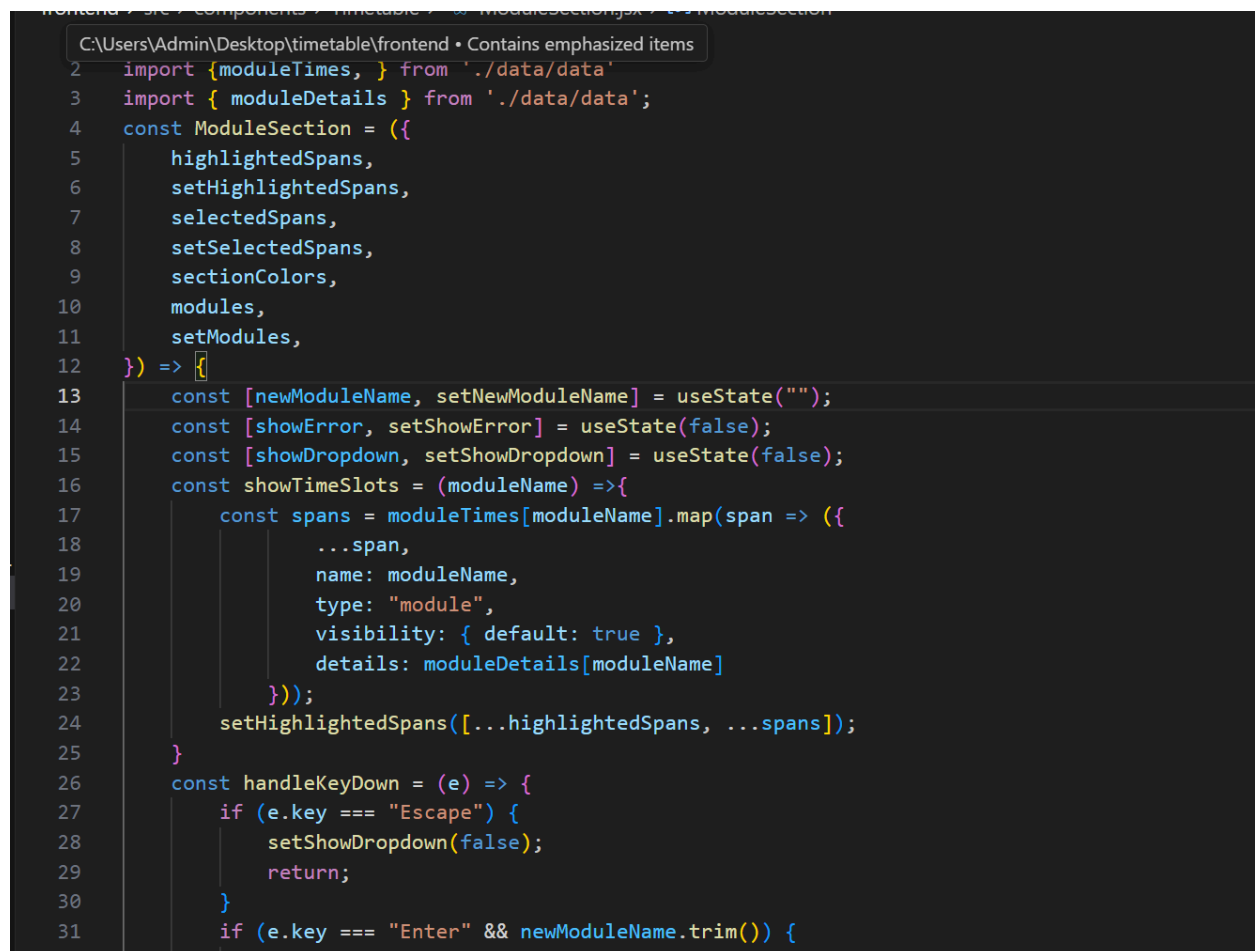
Timetable.jsx

3. Single Responsibility Principle (SRP)

Each component or module in our project has a **clearly defined responsibility**:

- `ModuleSection.jsx`: handles module data input, parsing, and rendering.
- `GeneralSection.jsx`: manages user-defined sections and their interactions.
- `useUI.js`: manages shared UI state like dropdowns, span selection, and color mapping.

This clarity in purpose allows each file to be **modified, tested, or extended independently**, which is particularly helpful during debugging or onboarding new developers.



```
C:\Users\Admin\Desktop\timetable\frontend • Contains emphasized items
2 import {moduleTimes, } from './data/data'
3 import { moduleDetails } from './data/data';
4 const ModuleSection = ({
5   highlightedSpans,
6   setHighlightedSpans,
7   selectedSpans,
8   setSelectedSpans,
9   sectionColors,
10  modules,
11  setModules,
12 }) => {
13   const [newModuleName, setNewModuleName] = useState("");
14   const [showError, setShowError] = useState(false);
15   const [showDropdown, setShowDropdown] = useState(false);
16   const showTimeSlots = (moduleName) =>{
17     const spans = moduleTimes[moduleName].map(span => ({
18       ...span,
19       name: moduleName,
20       type: "module",
21       visibility: { default: true },
22       details: moduleDetails[moduleName]
23     }));
24     setHighlightedSpans([...highlightedSpans, ...spans]);
25   }
26   const handleKeyDown = (e) => {
27     if (e.key === "Escape") {
28       setShowDropdown(false);
29       return;
30     }
31     if (e.key === "Enter" && newModuleName.trim()) {
```


ModuleSection.jsx

```
1  import { useState } from 'react';
2  import { days, time } from '../data/data';
3  const GeneralSection = ({
4    sectionName,
5    dropdownOpen_span,
6    setDropdownOpen_span,
7    selectedSpans,
8    setSelectedSpans,
9    highlightedSpans,
10   setHighlightedSpans,
11   sectionColors
12 }) => {
13   const [newSpanName, setNewSpanName] = useState("");
14   const [spanDay, setSpanDay] = useState(days[0]);
15   const [spanStart, setSpanStart] = useState(time[0]);
16   const [spanEnd, setSpanEnd] = useState(time[1]);
17
18   const handleAddSpan = (sectionType) => {
19     if(newSpanName.trim()){
20       const newSpan = {
21         day: spanDay,
22         start: spanStart,
23         end: spanEnd,
24         name: newSpanName.trim(),
25         type: sectionType.toLowerCase(),
26         visibility: {default: true},
27         details: ''
28       }
29       setSelectedSpans([...selectedSpans, newSpan]);
30       setHighlightedSpans([...highlightedSpans, newSpan]);
31     }
32     setNewSpanName('');
33     setDropdownOpen_span(false)
34   }
35
36   const sectionSpans = selectedSpans.filter(span => span.type === sectionType)
37   return (
38     <div className="feature-card">
```

GeneralSection.jsx

4. Configuration Management

We used **environment variables** to isolate environment-specific values:

- API endpoints,
- Auth tokens,
- Feature flags.

These are defined in a .env file and injected during build time using tools like dotenv.

This approach provides:

- **Security:** No sensitive data is hardcoded.
- **Flexibility:** Seamless switching between dev, staging, and production environments.
- **Maintainability:** Centralized configuration management reduces the risk of deployment issues.

```
const BE_URL = process.env.REACT_APP_BACKEND_URL;
```

```
const port = process.env.PORT;
app.listen(port, async() => {
  const data = await database.findOne({});
  console.log(`Server running on ${port}`);
});
```

```
const database = require('./database');
const database_chatbot = require('./database_chatbot');
const mongoose = require('mongoose');
const DATABASE_URL = process.env.DATABASE_URL
mongoose.connect(DATABASE_URL)
// .then(async () => {
```

```
const CHATBOT = process.env.REACT_APP_CHATBOT_URL
console.log(CHATBOT)
const response = await fetch(CHATBOT,{
  method: 'POST',
  headers:{
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    inputs: message.trim()
  }),
});
```

5. Conventional Commits

To keep our Git history **clean and meaningful**, we adopted the **Conventional Commits** standard.

Each commit starts with a prefix (feat:, fix:, refactor:), making it easier to:

- Understand commit intent at a glance,
- Track features and bug fixes,
- Generate changelogs automatically using tooling like commitlint or semantic-release.

This discipline has made team collaboration and release preparation smoother and more transparent.

Name	Last commit message	Last commit date
 GeneralSection.jsx	<Refactor, Feature> Rewrite entire frontend	now
 ModuleSection.jsx	<Refactor, Feature> Rewrite entire frontend	now
 Timetable.css	<Refactor, Feature> Rewrite entire frontend	now
 Timetable.jsx	<Refactor, Feature> Rewrite entire frontend	now

 src

<features, refactor> add module arrangem...

3 days ago

 .env

<fix bug> fix env file

3 days ago

Project Tracking with Github Project & Issues

Strong collaboration is crucial in any project, particularly when managing multiple tasks and interdependencies, no matter the team size. As a team of two, we quickly realized the value of structured project management to keep our development process clear and efficient.

From the outset, we implemented a system using GitHub Issues and Milestones to outline our timeline, delegate tasks, and monitor progress. This setup helped us maintain an organized workflow, with tasks, bugs, and deadlines clearly marked and regularly updated, ensuring transparency and keeping both of us aligned throughout the project.